

Pipeline de Datos del Macroentorno Ecuatoriano: Limpieza, Modelo Relacional y Dashboard Analítico

Martin Emanuel Ruiz Sanchez Carrera de Computación
 Universidad Técnica Particular de Loja
 Loja, Ecuador
 meruiz22@utpl.edu.ec

Resumen—

Index Terms—

I. INTRODUCCIÓN

El Tablero Macroentorno de la Universidad Técnica Particular de Loja consolida indicadores económicos y laborales de nueve fuentes públicas del Ecuador. Su actualización es manual y su pipeline carece de documentación reproducible. Cuando cambia un archivo fuente, el conocimiento para actualizarlo vive en personas, no en código.

El reto de Practicum 2.2 de 6to ciclo resuelve ese problema en siete semanas. El pipeline parte de los archivos crudos que entrega el equipo de RPA y termina en un dashboard con tres preguntas analíticas respondidas. El stack es Python, PostgreSQL y Power BI.

El dashboard responde tres preguntas concretas para la UTPL. P1 analiza la evolución económica ecuatoriana de los últimos 20 años con datos del BCE. P2 identifica los sectores y provincias con mayor actividad económica y empleo usando VAB, ENEMDU y el Censo 2022. P3 cruza bachilleres por provincia (MINEDUC) con empresas activas (Supercias) para detectar zonas con demanda potencial de educación superior; ese cruce alimenta directamente el Tablero de Segmentación de Marketing de la UTPL.

Construir ese pipeline requiere competencias que el mercado demanda con urgencia. El Future of Jobs Report 2025 proyecta que la demanda de especialistas en Big Data crece más del 100% entre 2025 y 2030 [1]. La UTPL forma a esos especialistas con datos reales, fuentes públicas y decisiones técnicas concretas.

El estado del arte responde cuatro preguntas vinculadas a los entregables del reto:

- **P1.** Qué arquitectura integra fuentes heterogéneas de datos públicos en un pipeline reproducible.
- **P2.** Cómo se diseña el modelo relacional antes de escribir código.
- **P3.** Qué stack técnico y prácticas de limpieza son críticos para cargar fuentes heterogéneas correctamente.
- **P4.** Cómo se integra el pipeline con un proceso de automatización y que anticipa ese patrón para 2025–2030.

La Fig. 1 resume la arquitectura completa del pipeline, con las tres capas del patrón medallón y su integración con el proceso de automatización RPA.

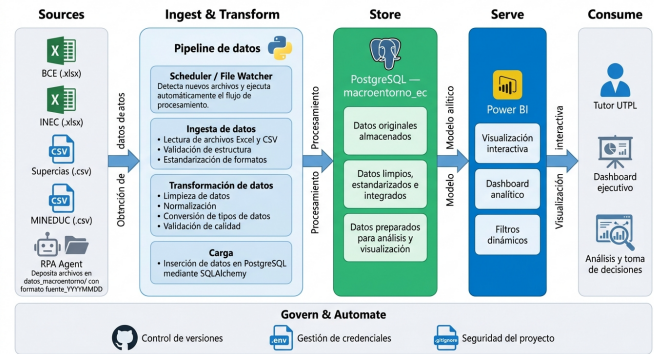


Figura 1. Arquitectura del pipeline del Macroentorno Ecuador. Bronze almacena los archivos crudos de RPA. Python los transforma y carga en PostgreSQL. Las vistas Gold alimentan la visualización en Power BI.

II. MARCO CONCEPTUAL

Esta sección define los conceptos que sostienen esa arquitectura, desde el almacenamiento de los datos hasta el pipeline que los mueve entre capas.

Data Warehouse. Repositorio central de datos históricos estructurados, integrados desde múltiples fuentes y optimizados para consultas analíticas [2]. Transforma los datos antes de cargarlos (ETL) y mantiene esquemas fijos.

Data Lake. Almacén de datos en formato crudo sobre almacenamiento de objetos de bajo costo [3]. Acepta datos estructurados, semi-estructurados y no estructurados sin transformación previa. Requiere metadatos activos para mantener la rastreabilidad de cada archivo.

Data Lakehouse. Arquitectura que une la flexibilidad del Data Lake con las garantías transaccionales del Data Warehouse en una sola plataforma [4]. Soporta cargas analíticas y de machine learning sin mover datos entre sistemas.

Arquitectura Medallón (Bronze–Silver–Gold). Patrón de tres zonas de calidad progresiva dentro de un Lakehouse [5]. Bronze guarda los datos en su estado original e inmutable. Silver aplica limpieza, tipado y normalización. Gold materializa vistas o tablas derivadas listas para el dashboard. En el reto, Bronze recibe los archivos de RPA, Silver los almacena en PostgreSQL con doce tablas y FK correctas, y Gold expone cuatro vistas para Power BI.

ETL (Extract, Transform, Load). Proceso que extrae datos, los transforma en un motor intermedio y los carga ya limpios en el destino [2]. Es el patrón clásico del Data Warehouse.

ELT (Extract, Load, Transform). Variante que carga los datos en crudo primero y transforma dentro del motor de destino [6]. El reto sigue ELT: cada módulo Python carga en PostgreSQL y las vistas Gold transforman ahí mismo.

Pipeline de datos. Secuencia automatizada de pasos que mueve y transforma datos desde las fuentes hasta el destino [7]. El pipeline del reto es reproducible: `pipeline.py` detecta archivos nuevos de RPA y los procesa sin intervención manual.

Modelo Entidad-Relación (ER). Representación gráfica de entidades, atributos y relaciones de un dominio [8].

III. TRABAJOS RELACIONADOS

La Tabla I sintetiza los trabajos que fundamentan las decisiones técnicas del reto.

Cuadro I
TRABAJOS RELACIONADOS

Ref.	Aporte	Conexión directa con el reto
[3]	Documenta el Lakehouse: storage y ACID	Bronze recibe archivos de RPA. Silver en PostgreSQL aplica ACID.
[4]	Sin metadatos y linaje el repositorio se degrada	Cada módulo documenta sus decisiones de limpieza: nulos, tipos y deduplicación.
[2]	Revisa la metodología dimensional de Kimball	Define <code>dim_tiempo</code> , <code>dim_geografia</code> y las doce tablas Silver con FK.
[6]	ELT: cada fuente propietaria de su lógica	<code>bce.py</code> , <code>inec.py</code> , <code>superacias.py</code> y <code>mineduc.py</code> encapsulan la limpieza de su fuente.
[9]	RPA optimiza ingestión; contratos críticos	El acuerdo de semana 4 fija subcarpetas, formato de nombre y canal de notificación.
[8]	Grafos ER son el estándar de BD modernas	El tutor aprueba el diagrama ER en semana 1 (10 % evaluación) antes de que el estudiante escriba código.
[10]	Párrafo interpretativo supera a la estética	6to ciclo exige un párrafo de análisis ejecutivo por página del dashboard.
[11]	Preparación de datos consume >70 % del tiempo	Las semanas 1 a 4 se dedican a limpieza y modelo. Power BI se conecta en semana 5.

Azzabi et al. [3] documentan la evolución del Lakehouse, formalizado originalmente por Armbrust et al. en CIDR 2021, hacia una arquitectura que combina almacenamiento económico con garantías transaccionales ACID. Jain et al. [12] compararon Delta Lake, Hudi e Iceberg: Delta Lake lidera en velocidad de carga; Iceberg ofrece mayor interoperabilidad. Harby y Zulkernine [4] confirman que documentar el linaje y los metadatos de cada archivo es lo que separa una arquitectura lakehouse madura de una improvisada. Nambiar y Mundra [2] revisan la metodología dimensional de Kimball en el contexto actual de data warehouse y data lake. Barret et al. [8] confirman que el diagrama ER sigue siendo la herramienta estándar para comunicar un diseño relacional antes de implementarlo. Machado et al. [6] describen el paso de ETL a ELT en el

contexto Data Mesh: cada fuente es propietaria de su lógica de limpieza. Bhardwaj et al. [9] demuestran que RPA escala el procesamiento de datos estructurados cuando los contratos de interfaz son precisos. Lai et al. [11] miden que la preparación de datos consume más del 70 % del tiempo en proyectos de BI. Tadakala et al. [10] demuestran que los dashboards con párrafo interpretativo por página superan en efectividad a los que priorizan la presentación visual.

IV. HERRAMIENTAS Y STACK TÉCNICO

El reto define un stack de tres capas: Python transforma, PostgreSQL almacena y Power BI visualiza. Antes de que ese stack procese un solo archivo, el reto exige diseñar el modelo relacional que sostiene las tablas Silver.

La Fig. 2 presenta ese modelo ER. Muestra las doce tablas con sus claves primarias y foráneas. `dim_tiempo` y `dim_geografia` se comparten entre las nueve fuentes. Las tablas de hechos (`fact_macro_anual`, `fact_empleo`, `fact_indicadores_diarios` y las de VAB, IEE, Superficies y MINEDUC) referencian esas dimensiones con FK.

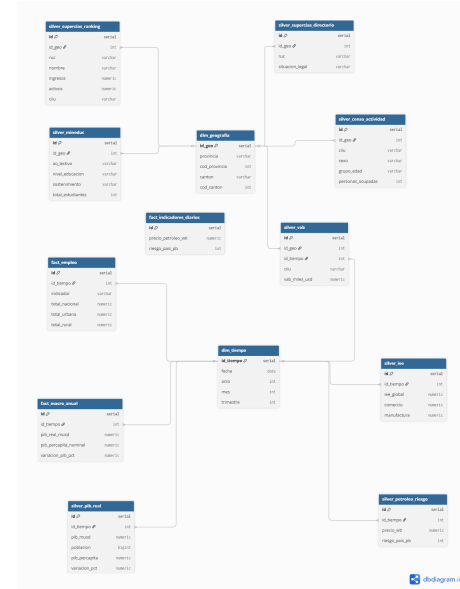


Figura 2. Modelo entidad-relación de la capa Silver del reto. `dim_tiempo` y `dim_geografia` son dimensiones compartidas. Cada tabla `fact_` referencia ambas con FK.

La Tabla II detalla el rol de cada herramienta en el pipeline.

Cuadro II
HERRAMIENTAS DEL STACK TÉCNICO

Herramienta	Uso
Python + pandas	Lenguaje y biblioteca para leer, limpiar y transformar datos tabulares dentro de scripts de extracción y carga.
PostgreSQL	Sistema de gestión de bases de datos relacionales, con soporte para vistas, particionamiento y tipos de dato JSONB.
Power BI Desktop	Herramienta de Microsoft para construir dashboards interactivos y conectarse a fuentes de datos mediante DirectQuery.
GitHub	Plataforma de control de versiones basada en Git, con repositorios públicos o privados y flujos de integración continua.
draw.io	Herramienta gratuita en línea para crear diagramas de arquitectura, flujos de proceso y modelos entidad-relación.

Python procesa cada fuente con un módulo independiente. `inec.py` aplica `melt()` para normalizar la estructura pivotada de ENEMDU. `mineduc.py` lee el AMIE con `encoding=latin-1` y `separador ;`, y filtra por bachillerato de último grado. `bce.py` convierte las fechas del IEE con `pd.to_datetime()` y carga los cinco archivos del BCE. PostgreSQL recibe las doce tablas Silver con FK correctas y expone cuatro vistas Gold. Power BI conecta esas vistas vía DirectQuery y construye las tres páginas del dashboard.

V. TENDENCIAS Y FUTURO

La Tabla III resume cinco tendencias del campo de datos para 2025–2030.

Cuadro III
TENDENCIAS 2025–2030 EN EL CAMPO DE DATOS

Tendencia	Descripción
IA Generativa en pipelines	Automatiza partes de la limpieza y la imputación de datos, aunque el juicio sobre nulos y tipos de dato sigue siendo del desarrollador [11].
DataOps continuo	Convierte la calidad de dato en parte del flujo de entrega, con pruebas dentro del pipeline en lugar de después de él [13].
Convergencia RPA-Datos	RPA ingesta fuentes que no ofrecen API, y el contrato de interfaz entre equipos se vuelve el punto crítico del proceso [9].
Data Mesh	Trata cada fuente como un producto de dato con un propietario y un nivel de servicio definidos [6].
Dashboards con análisis ejecutivo	Un párrafo interpretativo por página comunica mejor las decisiones que un dashboard puramente visual [10].

La IA Generativa automatiza partes de la limpieza, pero la decisión de cómo normalizar ENEMDU con `melt()` o cómo tratar el primer año de variación nula del PIB real requiere juicio del estudiante [1]. Foidl et al. [13] identifican 41 factores que influyen en la calidad de un pipeline de datos y ubican la etapa de limpieza como la principal fuente de errores, lo que confirma que DataOps convierte la calidad de dato en parte del flujo de entrega. El WEF proyecta más del 100 % de crecimiento en demanda de especialistas en datos para 2030 [1].

Las habilidades que el reto desarrolla, diseño ER, limpieza de fuentes reales y storytelling analítico, son las que el mercado demanda.

VI. CONCLUSIONES

VI-A. P1. Arquitectura para integrar fuentes heterogéneas en un pipeline reproducible

La arquitectura Lakehouse con patrón medallón Bronze–Silver–Gold organiza el pipeline en tres zonas de calidad progresiva [3], [5] (Fig. 1). Bronze recibe los archivos de las fuentes públicas sin modificarlos. Python los transforma y los carga en tablas relacionales en PostgreSQL. Las vistas Gold alimentan Power BI con indicadores listos para el dashboard. El repositorio GitHub con módulos versionados garantiza que cualquier miembro del equipo reproduce el resultado. Harby y Zulkernine [4] confirman que documentar el linaje y los metadatos de cada decisión de limpieza es lo que diferencia un repositorio útil de uno inutilizable.

VI-B. P2. Modelo relacional diseñado antes de escribir código

La metodología dimensional de cuatro pasos (proceso de negocio, granularidad, dimensiones y métricas), revisada por Nambiar y Mundra [2] en el contexto actual de data warehouse y data lake, ordena el diseño del modelo relacional. El esquema resultante tiene dimensiones compartidas entre las fuentes y tablas de hechos con FK correctas (Fig. 2). Barret et al. [8] confirman que el diagrama ER comunica el diseño antes de que el código exista. El tutor aprueba ese diagrama al inicio del proyecto. Cada error detectado en el diagrama es un error que no aparece en la base de datos.

VI-C. P3. Stack técnico y prácticas de limpieza para fuentes heterogéneas

Python procesa cada fuente con un módulo independiente que encapsula su lógica de limpieza: renombrado de columnas, tratamiento de nulos, conversión de tipos y normalización de fechas. PostgreSQL almacena las tablas relacionales y expone vistas Gold listas para DirectQuery. Power BI construye tres páginas analíticas que responden las preguntas P1, P2 y P3 del dashboard. Lai et al. [11] miden que más del 70 % del tiempo de un proyecto de BI va a preparación de datos. Dedicar la mayor parte del trabajo a la limpieza y al modelo antes de conectar la herramienta de visualización es la práctica que distingue proyectos de BI exitosos de los que fallan en producción.

VI-D. P4. Integración con automatización y perspectiva 2025–2030

La integración entre el pipeline de datos y un proceso de automatización requiere un contrato de interfaz preciso: nombre de carpeta, formato de archivo y canal de notificación. Bhardwaj et al. [9] demuestran que la automatización escala cuando esos contratos son precisos. El pipeline detecta los archivos nuevos y los procesa sin intervención manual (Fig. 1). Para 2030, el Data Mesh y DataOps consolidan un modelo donde cada fuente es un producto de dato con propietario y SLA [6]. El estudiante que termina este reto practica ese modelo con datos reales desde el primer día.

REFERENCIAS

- [1] World Economic Forum, *Future of Jobs Report 2025*. Ginebra: WEF, 2025. <https://www.weforum.org/publications/the-future-of-jobs-report-2025/>
- [2] A. Nambiar y D. Mundra, “An overview of data warehouse and data lake in modern enterprise data management,” *Big Data Cogn. Comput.*, vol. 6, no. 4, p. 132, 2022. <https://doi.org/10.3390/bdcc6040132>
- [3] S. Azzabi, Z. Alfughi y A. Ouda, “Data lakes: A survey of concepts and architectures,” *Computers*, vol. 13, no. 7, p. 183, 2024. <https://doi.org/10.3390/computers13070183>
- [4] A. A. Harby y F. Zulkernine, “From data warehouse to lakehouse: A comparative review,” en *Proc. IEEE Int. Conf. Big Data*, Osaka, Japón, 2022, pp. 389–395. <https://doi.org/10.1109/BigData55660.2022.10020719>
- [5] Databricks, “What is medallion lakehouse architecture?” *Microsoft Azure Docs*, 2024. <https://learn.microsoft.com/en-us/azure/databricks/lakehouse/medallion>
- [6] I. A. Machado, C. Costa y M. Y. Santos, “Data mesh: Concepts and principles of a paradigm shift in data architectures,” *Procedia Comput. Sci.*, vol. 196, pp. 263–271, 2022. <https://doi.org/10.1016/j.procs.2021.12.033>
- [7] J. Naso y C. Padden, *Data Platform Fundamentals*. Dagster Labs, 2024. <https://dagster.io/blog/data-platform-fundamentals>
- [8] N. Barret et al., “Entity/Relationship graphs: Principled design, modeling, and data integrity management,” *Proc. ACM Manag. Data*, vol. 3, no. 1, 2025. <https://doi.org/10.1145/3709690>
- [9] V. Bhardwaj, A. Noonina, S. Chaurasia, M. Kumar, A. Rashid y M. T. Ben Othman, “Optimizing structured data processing through robotic process automation,” *J. Eur. Syst. Autom.*, vol. 57, no. 5, pp. 1523–1530, 2024. <https://doi.org/10.18280/jesa.570528>
- [10] L. Tadakala et al., “Beyond visualization: Building decision intelligence through iterative dashboard refinement,” arXiv:2510.27572, 2025. <https://arxiv.org/pdf/2510.27572>
- [11] E. Lai, Y. He y S. Chaudhuri, “Auto-Prep: Holistic prediction of data preparation steps for self-service BI,” *Proc. VLDB Endow.*, vol. 14, no. 7, 2025. arXiv:2504.11627.
- [12] P. Jain, P. Kraft, C. Power, T. Das, I. Stoica y M. Zaharia, “Analyzing and comparing lakehouse storage systems,” en *Proc. 13th CIDR*, 2023. <https://www.cidrdb.org/cidr2023/papers/p92-jain.pdf>
- [13] H. Foidl, V. Golendukhina, R. Ramler y M. Felderer, “Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers,” *J. Syst. Softw.*, vol. 207, p. 111855, 2024. <https://doi.org/10.1016/j.jss.2023.111855>